Chapter 13: Distributed Data
Part 3 - Saga Pattern

Quản lý distributed transaction bằng chuỗi local transaction kèm compensation

Stack: Spring Boot 3.x, RabbitMQ, Kafka, PostgreSQL

NỘI DUNG PHẦN NÀY

1. Vấn đề: Distributed Transaction trong Microservices
    - Bài toán kinh điển: Đặt hàng E-commerce
    - Ý tưởng cốt lõi của Saga Pattern
    - Bài toán trong TalentHub
2. Saga Pattern: định nghĩa và nguyên tắc
    - Nguồn gốc và định nghĩa
    - Compensating Transaction
3. Choreography vs Orchestration
    - Choreography-based Saga
    - Orchestration-based Saga
    - So sánh và lựa chọn
4. Application Submission Saga trong TalentHub
    - Thiết kế luồng happy path
    - Thiết kế compensation chain
5. Triển khai Choreography Saga cho TalentHub
6. Xử lý failure và compensation
7. Các lưu ý thiết kế Saga trong thực tế
8. Tóm tắt và câu hỏi ôn tập

01

Vấn đề: Distributed Transaction trong Microservices

1.1. Bài toán kinh điển: Đặt hàng E-commerce

Trong kiến trúc microservices, một nghiệp vụ thường trải dài qua nhiều service khác nhau, mỗi service có database riêng. Ví dụ, khi đặt một đơn hàng:

1. **Order service** tạo đơn hàng
2. **Payment service** trừ tiền khách hàng
3. **Inventory service** trừ số lượng tồn kho
4. **Shipping service** tạo lệnh vận chuyển

Tuy nhiên, do mỗi service quản lý cơ sở dữ liệu riêng, hệ thống không thể sử dụng transaction truyền thống để đảm bảo tính toàn vẹn cho toàn bộ các bước. Nếu bước 3 (trừ tồn kho) thất bại do hết hàng, khoản tiền đã bị trừ ở bước 2 cần một cơ chế để hoàn trả.

Đây chính là bài toán mà **Saga Pattern** giải quyết: đảm bảo tính nhất quán dữ liệu xuyên suốt nhiều service, mà không cần distributed transaction (kiểu 2PC - vốn phức tạp, khóa tài nguyên lâu, không phù hợp cho hệ thống phân tán quy mô lớn).

1.2. Ý tưởng cốt lõi của Saga Pattern

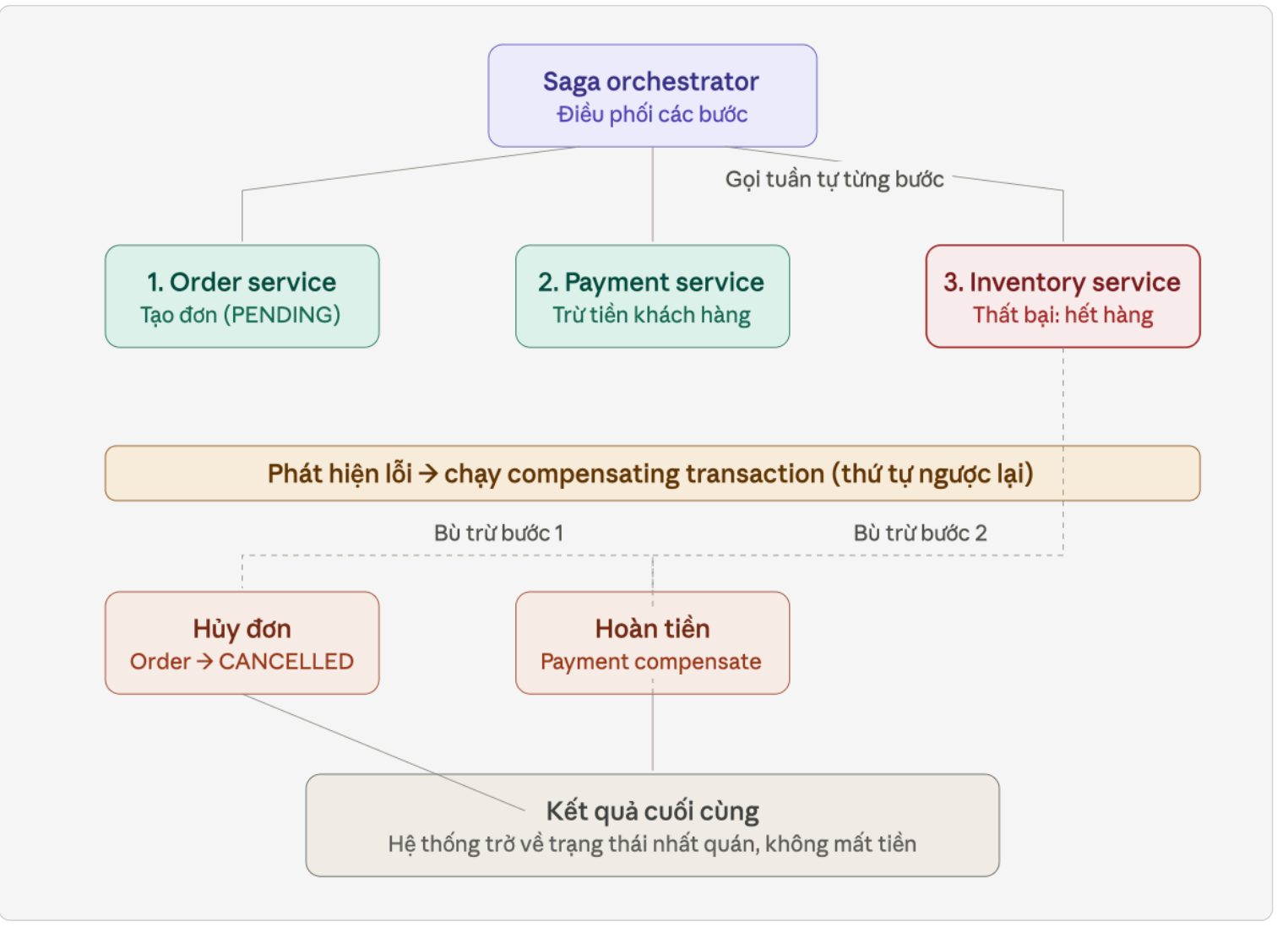
Một saga là một chuỗi các **local transaction** (transaction cục bộ trong từng service). Mỗi bước:

- Nếu **thành công**, hệ thống sẽ kích hoạt bước tiếp theo của quy trình.
- Nếu **thất bại**, luồng Saga sẽ kích hoạt các **compensating transaction** (giao dịch bù trừ) để hoàn tác những gì các bước trước đã thực hiện trước đó (tương tự khái niệm "undo" theo thứ tự ngược lại).

Có 2 cách triển khai chính:

- **Choreography (biên đạo):** các service tự lắng nghe event của nhau và tự quyết định hành động tiếp theo, không có nhạc trưởng.
- **Orchestration (điều phối):** có 1 "nhạc trưởng" trung tâm (Saga Orchestrator) ra lệnh cho từng service làm gì, theo dõi toàn bộ tiến trình.

Hãy xem sơ đồ minh họa kiểu Orchestration - dễ hình dung hơn cho người mới:


Saga orchestrator
Điều phối các bước
Gọi tuần tự từng bước
1. Order service
Tạo đơn (PENDING)
2. Payment service
Trừ tiền khách hàng
3. Inventory service
Thất bại: hết hàng
Phát hiện lỗi → chạy compensating transaction (thứ tự ngược lại)
Bù trừ bước 1
Bù trừ bước 2
Hủy đơn
Order → CANCELLED
Hoàn tiền
Payment compensate
Kết quả cuối cùng
Hệ thống trở về trạng thái nhất quán, không mất tiền

GIẢI THÍCH VÍ DỤ TRONG SƠ ĐỒ

Kịch bản: khách đặt một đơn hàng, nhưng bước kiểm tra kho lại phát hiện hết hàng.

1. Order service tạo đơn với trạng thái `PENDING`
2. Payment service trừ tiền khách hàng thành công
3. Inventory service cố gắng trừ tồn kho nhưng thất bại (hết hàng)

Vì bước 3 thất bại, saga orchestrator phải "quay ngược lại" - chạy các compensating transaction theo thứ tự ngược với thứ tự đã thực hiện:

- Hoàn tiền lại cho khách (bù trừ cho bước 2)
- Cập nhật đơn hàng thành `CANCELLED` (bù trừ cho bước 1)

> LƯU Ý
>
> Bước 3 (Inventory) không cần bù trừ vì nó chưa từng thành công - không có gì để hoàn tác.

1.3. Bài toán trong TalentHub

Tương tự như E-commerce, trong monolith TalentHub, khi candidate nộp đơn, tất cả logic nằm trong một method với một `@Transactional`:

```java
@Transactional
public void submitApplication(SubmitRequest req) {
    // 1. Validate job con mo
    Job job = jobRepo.findById(req.jobId());

    // 2. Kiem tra candidate khong trung
    Candidate candidate = candidateRepo.findById(req.candidateId());

    // 3. Tao application
    Application app = new Application(candidate, job);
    applicationRepo.save(app);

    // 4. Gui email xac nhan
    emailService.sendConfirmation(candidate.getEmail(), app);

    // 5. Kich hoat parse CV
    cvParserService.parse(candidate.getCvFile());
}
// Neu bat ky buoc nao fail, toan bo rollback.
```

Với microservices, mỗi bước trên thuộc một service khác, có database riêng. Không thể dùng một `@Transactional` bao trùm tất cả. Vậy làm sao đảm bảo tính nhất quán khi một bước trung gian thất bại?

Tại sao 2PC không phù hợp?

Two-Phase Commit (2PC) yêu cầu một transaction coordinator khóa tài nguyên trên tất cả participant cho đến khi tất cả sẵn sàng commit. Trong microservices:

- **Giảm availability:** Nếu một participant chậm hoặc down, toàn bộ transaction bị block (theo CAP theorem, 2PC chọn Consistency trên Availability).
- **Không scale:** Lock tài nguyên cross-service tạo bottleneck nghiêm trọng khi hệ thống mở rộng.
- **Coupling chặt:** Tất cả participant phải support XA protocol. Hầu hết message broker và NoSQL database không hỗ trợ.

Richardson kết luận: *"2PC is not an option for modern microservice applications"* (Richardson, 2018, Ch. 4).

02

Saga Pattern: định nghĩa và nguyên tắc

2.1. Nguồn gốc và định nghĩa

Khái niệm Saga được đề xuất lần đầu bởi Hector Garcia-Molina và Kenneth Salem trong bài báo "Sagas" (ACM SIGMOD, 1987). Ban đầu nó được thiết kế cho long-lived transaction trong database đơn lẻ, sau đó được Chris Richardson áp dụng vào microservices.

> *"A saga is a sequence of local transactions. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga. If a local transaction fails, the saga executes a series of compensating transactions that undo the changes made by the preceding local transactions."*
> *- Nguồn: Chris Richardson, Microservices Patterns, Ch. 4*

Các đặc tính quan trọng:

- **Mỗi bước là một local transaction:** Mỗi service chỉ commit vào database của chính nó. Không có global lock.
- **Eventual consistency:** Dữ liệu giữa các service nhất quán theo thời gian, không tức thì.
- **Compensation thay vì rollback:** Không thể rollback một transaction đã commit ở service khác. Thay vào đó, thực thi một "compensating transaction" ngược lại để hoàn tác tác động.

2.2. Compensating Transaction

Compensating transaction (giao dịch bù) là một thao tác ngược lại với thao tác đã thực hiện, nhằm hoàn tác tác động của một bước trong saga khi bước sau đó thất bại.

BƯỚC GỐC	COMPENSATING TRANSACTION	GHI CHÚ
Tạo Application (status = NEW)	Đánh dấu Application = FAILED hoặc xóa	Không xóa vật lý, đánh dấu để audit
Gửi email xác nhận	Gửi email sửa lỗi / thông báo hủy	Email đã gửi không thể thu hồi; gửi email bổ sung
Parse CV (tạo parsed data)	Xóa parsed data, đánh dấu "invalid"	Hoặc giữ lại cho manual review
Trừ credit ứng viên	Hoàn credit	Ví dụ mở rộng, không có trong TalentHub

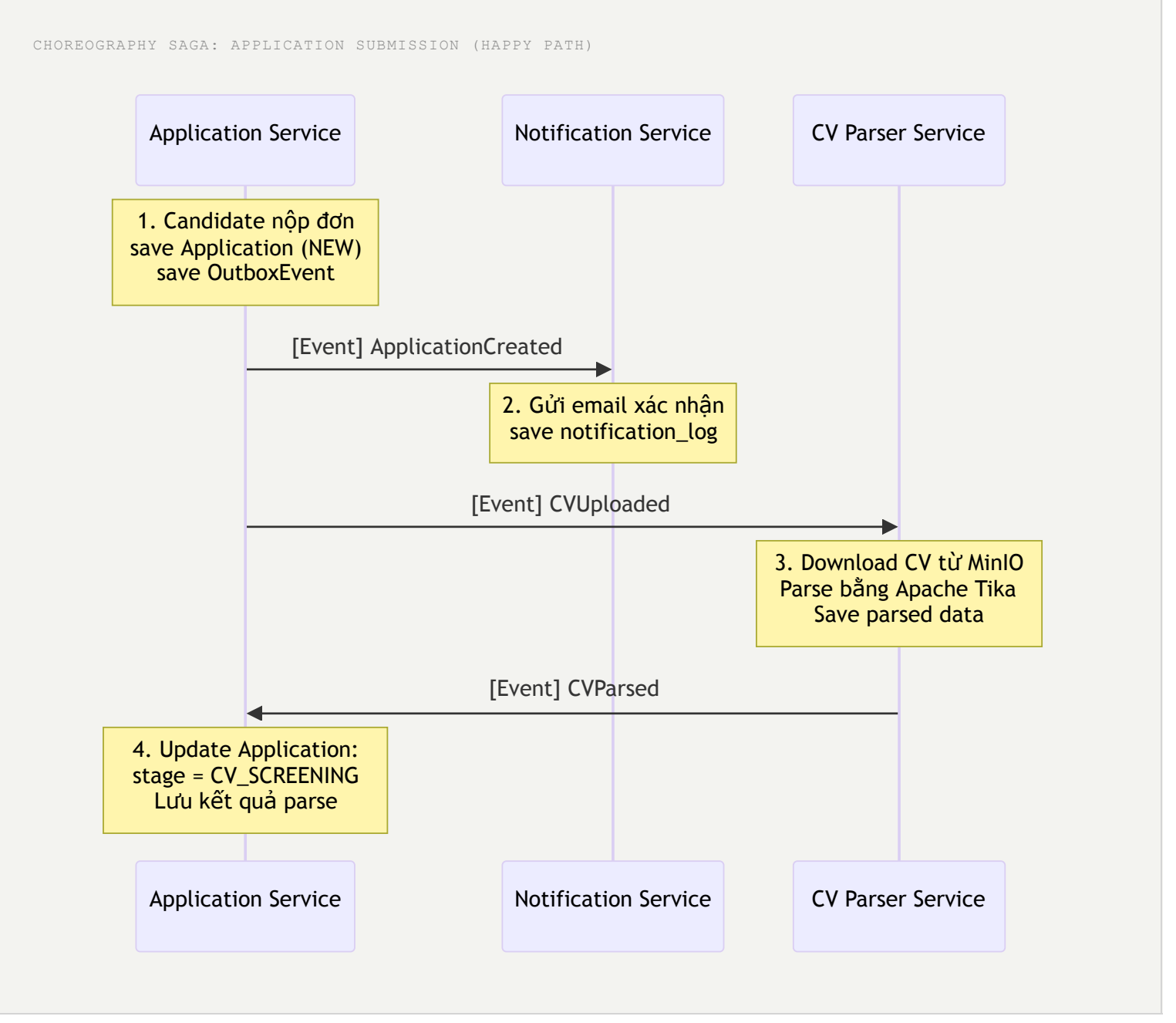
> **KHÔNG PHẢI MỌI BƯỚC ĐỀU CẦN COMPENSATION**
>
> Richardson chia các bước saga thành ba loại:
>
> - **Compensatable transaction:** Bước có thể bị undo (ví dụ: tạo application).
> - **Pivot transaction:** Bước quyết định saga thành công hay thất bại. Không cần compensation vì nó là "điểm không quay lại" (ví dụ: commit final).
> - **Retriable transaction:** Bước sau pivot, luôn phải thành công (có thể retry). Không cần compensation vì saga đã qua pivot (ví dụ: gửi email thông báo thành công).

03

Choreography vs Orchestration

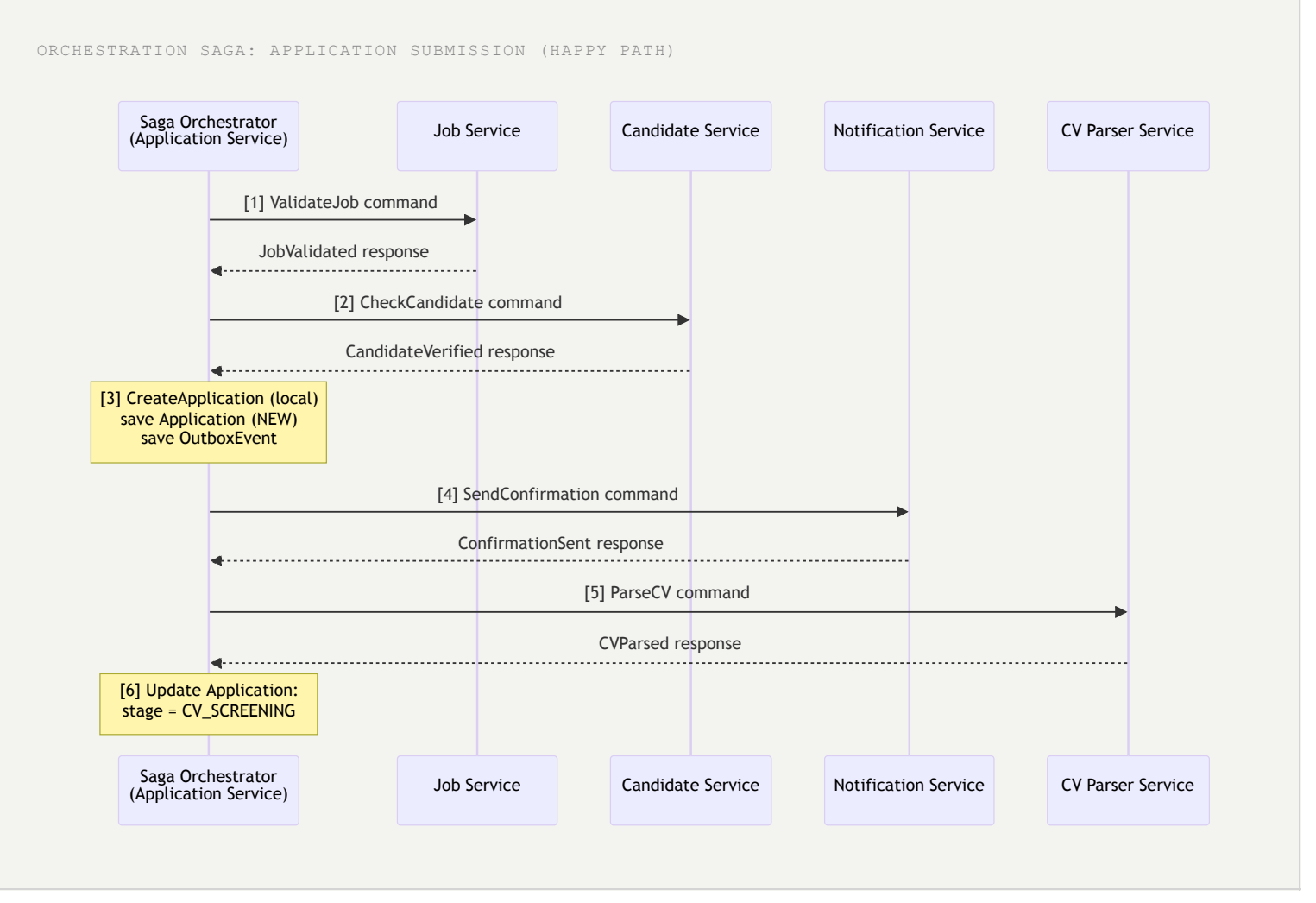
3.1. Choreography-based Saga

Trong choreography, **không có coordinator trung tâm**. Mỗi service lắng nghe event từ service trước, thực hiện local transaction của mình, rồi publish event cho service tiếp theo. Tương tự như một vũ đạo (choreography): Mỗi thành phần tự nhận biết hành động tiếp theo dựa trên các sự kiện phát sinh từ các thành phần khác.


CHOREOGRAPHY SAGA: APPLICATION SUBMISSION (HAPPY PATH)
Application Service
Notification Service
CV Parser Service
1. Candidate nộp đơn
save Application (NEW)
save OutboxEvent
[Event] ApplicationCreated
2. Gửi email xác nhận
save notification_log
[Event] CVUploaded
3. Download CV từ MinIO
Parse bằng Apache Tika
Save parsed data
[Event] CVParsed
4. Update Application:
stage = CV_SCREENING
Lưu kết quả parse
Application Service
Notification Service
CV Parser Service

3.2. Orchestration-based Saga

Trong orchestration, có một **Saga Orchestrator** (thường nằm trong service khởi tạo saga) điều phối toàn bộ luồng. Orchestrator gửi command tới từng service theo thứ tự, đợi response, rồi quyết định bước tiếp theo. Mô hình này tương tự như một nhạc trưởng (orchestrator) điều khiển một dàn nhạc.


ORCHESTRATION SAGA: APPLICATION SUBMISSION (HAPPY PATH)
Saga Orchestrator
(Application Service)
Job Service
Candidate Service
Notification Service
CV Parser Service
[1] ValidateJob command
JobValidated response
[2] CheckCandidate command
CandidateVerified response
[3] CreateApplication (local)
save Application (NEW)
save OutboxEvent
[4] SendConfirmation command
ConfirmationSent response
[5] ParseCV command
CVParsed response
[6] Update Application:
stage = CV_SCREENING
Saga Orchestrator
(Application Service)
Job Service
Candidate Service
Notification Service
CV Parser Service

3.3. So sánh và lựa chọn

TIÊU CHÍ	CHOREOGRAPHY	ORCHESTRATION
Coupling	Loose: mỗi service chỉ biết event, không biết ai consume	Tighter: orchestrator biết tất cả participant
Dễ hiểu	Khó theo dõi khi có nhiều bước (event chain dài)	Dễ theo dõi: logic tập trung trong orchestrator
Single point of failure	Không có: decentralized	Orchestrator là SPOF (cần HA)
Thêm bước mới	Thêm consumer mới, không sửa service cũ	Phải sửa orchestrator
Cyclic dependency	Dễ xảy ra nếu không cẩn thận	Không xảy ra: luồng một chiều
Phù hợp	Saga đơn giản, 2 tới 4 bước	Saga phức tạp, nhiều bước, nhiều nhánh rẽ

> LỰA CHỌN CỦA TALENTHUB
>
> Trong khóa học này, chúng ta triển khai **Choreography-based Saga** cho luồng Application Submission vì: (1) luồng có 3 tới 4 bước, đủ đơn giản cho choreography; (2) sinh viên hiểu rõ event-driven communication đã học ở Chapter 11-12; (3) không cần tạo thêm orchestrator component. Trong dự án thực tế với luồng phức tạp hơn (ví dụ: luồng Offer có 7+ bước), nên cân nhắc orchestration.

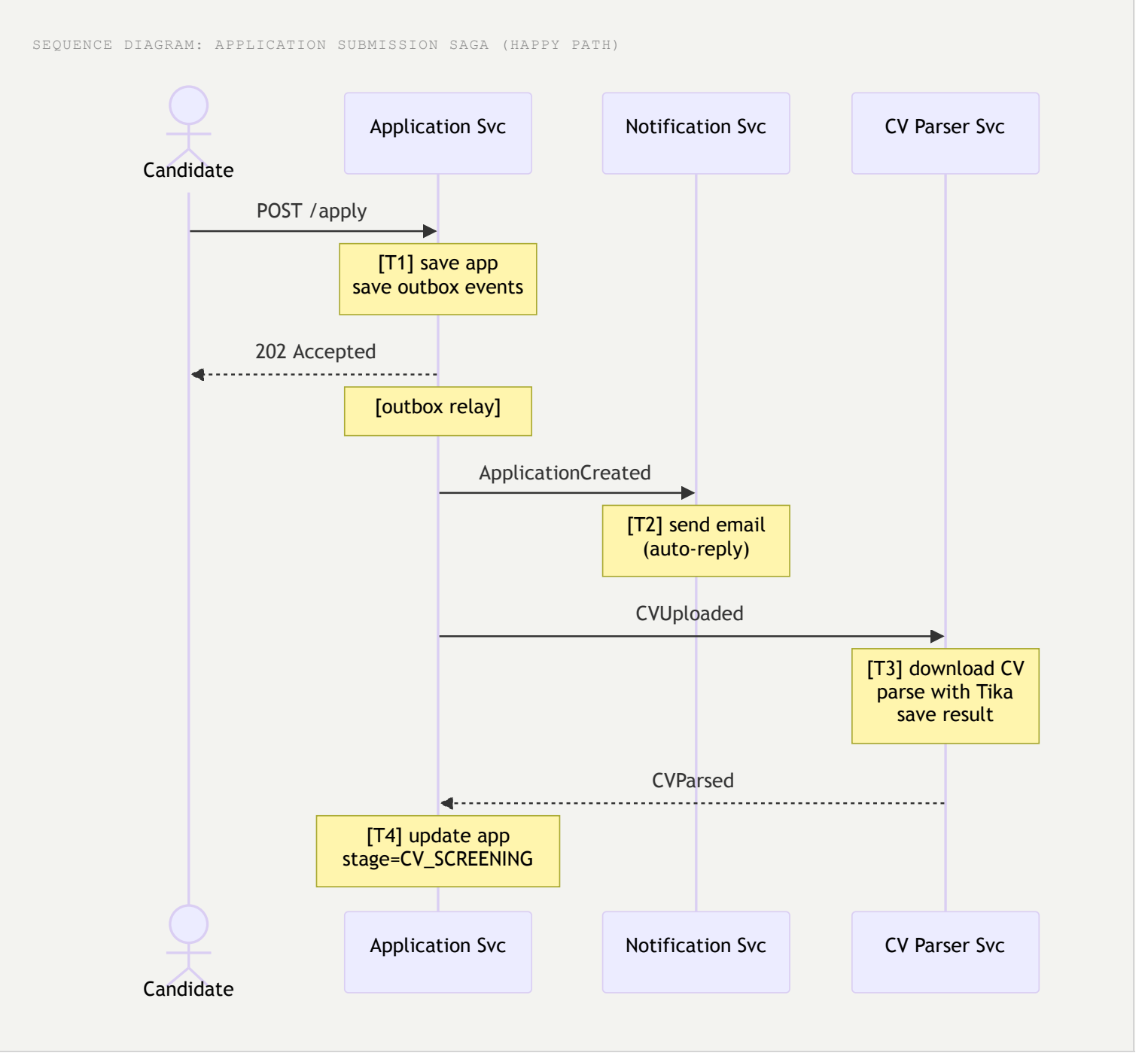
04

Application Submission Saga trong TalentHub

4.1. Thiết kế luồng happy path

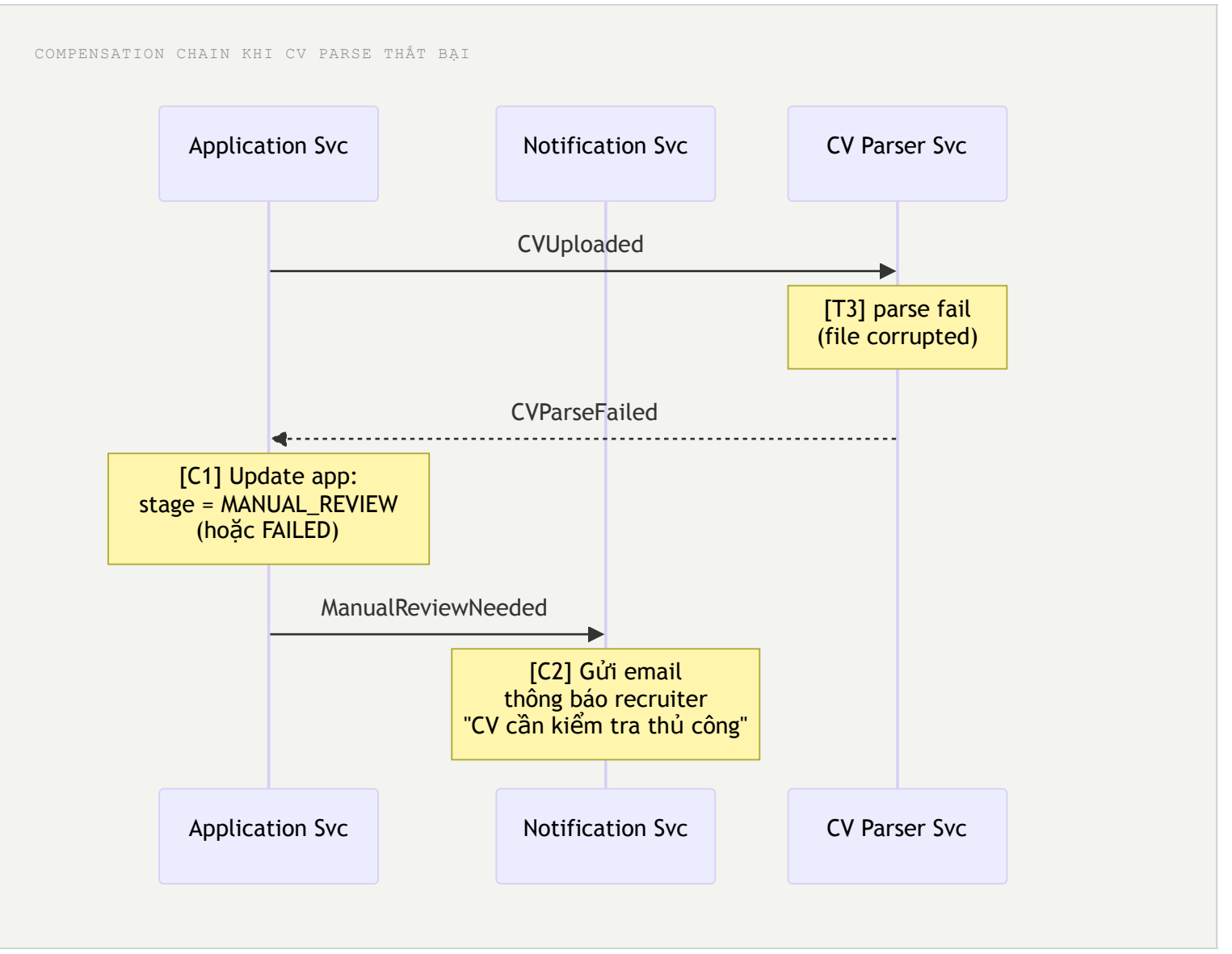
Luồng nộp đơn (Application Submission) trong TalentHub gồm các bước:

BƯỚC	SERVICE	LOCAL TRANSACTION	EVENT PUBLISH	LOẠI
T1	Application Service	Validate job (Feign), validate candidate (Feign), tạo Application (NEW), ghi outbox	`ApplicationCreated`, `CVUploaded`	Compensatable
T2	Notification Service	Consume `ApplicationCreated`, gửi email auto-reply, ghi notification_log	`NotificationSent` (optional)	Retriable
T3	CV Parser Service	Consume `CVUploaded`, download CV từ MinIO, parse bằng Tika, lưu kết quả	`CVParsed` hoặc `CVParseFailed`	Pivot
T4	Application Service	Consume `CVParsed`, update Application stage = CV_SCREENING	(End of saga)	Retriable


SEQUENCE DIAGRAM: APPLICATION SUBMISSION SAGA (HAPPY PATH)
Candidate
Application Svc
Notification Svc
CV Parser Svc
POST /apply
[T1] save app
save outbox events
202 Accepted
[outbox relay]
ApplicationCreated
[T2] send email
(auto-reply)
CVUploaded
[T3] download CV
parse with Tika
save result
CVParsed
[T4] update app
stage=CV_SCREENING
Candidate
Application Svc
Notification Svc
CV Parser Svc

4.2. Thiết kế compensation chain

Khi T3 (CV parsing) thất bại (file hỏng, format không hỗ trợ), CV Parser Service publish event `CVParseFailed`. Application Service nhận event này và thực hiện compensation:


COMPENSATION CHAIN KHI CV PARSE THẤT BẠI
Application Svc
Notification Svc
CV Parser Svc
CVUploaded
[T3] parse fail
(file corrupted)
CVParseFailed
[C1] Update app:
stage = MANUAL_REVIEW
(hoặc FAILED)
ManualReviewNeeded
[C2] Gửi email
thông báo recruiter
"CV cần kiểm tra thủ công"
Application Svc
Notification Svc
CV Parser Svc

Lưu ý rằng compensation ở đây không phải xóa application (vì candidate đã nộp đơn hợp lệ), mà là chuyển sang trạng thái cần kiểm tra thủ công. Đây là **semantic compensation**: thay vì rollback hoàn toàn, hệ thống chuyển sang trạng thái an toàn để con người can thiệp.

> **SEMANTIC COMPENSATION TRONG THỰC TẾ**
>
> Trong hệ thống tuyển dụng thực tế, rất hiếm khi compensation là "xóa hoàn toàn". Thay vào đó, hệ thống chuyển sang trạng thái "cần xem xét thủ công" (manual review) hoặc "chờ xử lý" (pending). Lý do: (1) audit trail quan trọng hơn data cleanup; (2) hành động của candidate (nộp đơn) là có thật, không thể "undo" từ góc nhìn nghiệp vụ; (3) recruiter cần biết có issue để follow up.

05

Triển khai Choreography Saga cho TalentHub

5.1. Event definitions (shared module)

Các event class nên đặt trong module `shared/common-events` để cả producer và consumer cùng dùng:

shared/common-events/src/.../events/ApplicationCreatedEvent.java

```java
package com.talenthub.events;

import java.util.UUID;

public record ApplicationCreatedEvent(
    UUID eventId,           // ID cua outbox event (dung cho idempotent check)
    UUID applicationId,
    UUID candidateId,
    UUID jobId,
    String candidateName,
    String candidateEmail
) {}
```

shared/common-events/src/.../events/CVUploadedEvent.java

```java
package com.talenthub.events;

import java.util.UUID;

public record CVUploadedEvent(
    UUID eventId,
    UUID applicationId,
    UUID candidateId,
    String cvFileUrl        // URL file CV tren MinIO
) {}
```

shared/common-events/src/.../events/CVParsedEvent.java

```java
package com.talenthub.events;

import java.util.UUID;

public record CVParsedEvent(
    UUID eventId,
    UUID applicationId,
    UUID candidateId,
    String fullName,
    String email,
    String phone,
    String summary          // tom tat ket qua parse
) {}
```

shared/common-events/src/.../events/CVParseFailedEvent.java

```java
package com.talenthub.events;

import java.util.UUID;

public record CVParseFailedEvent(
    UUID eventId,
    UUID applicationId,
    UUID candidateId,
    String reason           // ly do that bai: "Unsupported format", "File corrupted"
) {}
```

5.2. Application Service: publish events qua Outbox

Khi candidate nộp đơn kèm CV, Application Service ghi hai outbox event trong cùng transaction:

SubmitApplicationUseCase.java (voi Outbox + Saga events)

```java
@Transactional
public UUID execute(SubmitApplicationCommand cmd) {

    // ... validate job, candidate, duplicate check ...

    // Luu application
    Application app = Application.submit(cmd.candidateId(), cmd.jobId());
    repo.save(app);

    // Outbox event 1: ApplicationCreated (RabbitMQ, cho Notification Service)
    outboxRepo.save(OutboxEvent.create(
        "Application", app.getId(),
        "ApplicationCreated",
        toJson(new ApplicationCreatedEvent(
            null, app.getId(), cmd.candidateId(), cmd.jobId(),
            candidate.fullName(), candidate.email()
        ))
    ));

    // Outbox event 2: CVUploaded (Kafka, cho CV Parser Service)
    if (cmd.cvFileUrl() != null) {
        outboxRepo.save(OutboxEvent.create(
            "Application", app.getId(),
            "CVUploaded",
            toJson(new CVUploadedEvent(
                null, app.getId(), cmd.candidateId(), cmd.cvFileUrl()
            ))
        ));
    }

    return app.getId();
}
```

5.3. CV Parser Service: consume CVUploaded, publish CVParsed/CVParseFailed

CvParserEventListener.java (CV Parser Service)

```java
@Component
@RequiredArgsConstructor
@Slf4j
public class CvParserEventListener {

    private final MinioClient minioClient;
    private final TikaParser tikaParser;
    private final CvParseResultRepository resultRepo;
    private final KafkaTemplate<String, String> kafkaTemplate;
    private final ObjectMapper objectMapper;

    @KafkaListener(topics = "cv.uploaded", groupId = "cv-parser-group")
    public void onCvUploaded(String message) {
        CVUploadedEvent event = parseEvent(message, CVUploadedEvent.class);

        try {
            // 1. Download CV tu MinIO
            InputStream cvStream = minioClient.getObject(
                GetObjectArgs.builder()
                    .bucket("cv-uploads")
                    .object(event.cvFileUrl())
                    .build()
            );

            // 2. Parse bang Apache Tika
            ParsedContent content = tikaParser.parse(cvStream);

            // 3. Luu ket qua
            CvParseResult result = CvParseResult.success(
                event.applicationId(), event.candidateId(),
                content.fullName(), content.email(),
                content.phone(), content.summary()
            );
            resultRepo.save(result);

            // 4. Publish CVParsed event
            kafkaTemplate.send("cv.parsed", toJson(new CVParsedEvent(
                UUID.randomUUID(), event.applicationId(), event.candidateId(),
                content.fullName(), content.email(),
                content.phone(), content.summary()
            )));

            log.info("CV parsed successfully for application={}",
                     event.applicationId());

        } catch (Exception ex) {
            log.error("CV parse failed for application={}: {}",
                      event.applicationId(), ex.getMessage());

            // Publish CVParseFailed event (compensation trigger)
            kafkaTemplate.send("cv.parse.failed", toJson(new CVParseFailedEvent(
                UUID.randomUUID(), event.applicationId(),
                event.candidateId(), ex.getMessage()
            )));
        }
    }
}
```

5.4. Application Service: consume CVParsed / CVParseFailed

SagaEventListener.java (Application Service)

```java
@Component
@RequiredArgsConstructor
@Slf4j
public class SagaEventListener {

    private final ApplicationRepository applicationRepo;

    /**
     * Happy path: CV parsed thanh cong, chuyen stage.
     */
    @KafkaListener(topics = "cv.parsed", groupId = "application-saga-group")
    @Transactional
    public void onCvParsed(String message) {
        CVParsedEvent event = parseEvent(message, CVParsedEvent.class);

        Application app = applicationRepo.findById(event.applicationId())
            .orElseThrow(() -> new ApplicationNotFoundException(event.applicationId()));

        app.advanceStage(PipelineStage.CV_SCREENING);
        applicationRepo.save(app);

        log.info("Saga completed: application={} advanced to CV_SCREENING",
                 event.applicationId());
    }

    /**
     * Compensation: CV parse that bai, chuyen sang manual review.
     */
    @KafkaListener(topics = "cv.parse.failed", groupId = "application-saga-group")
    @Transactional
    public void onCvParseFailed(String message) {
        CVParseFailedEvent event = parseEvent(message, CVParseFailedEvent.class);

        Application app = applicationRepo.findById(event.applicationId())
            .orElseThrow(() -> new ApplicationNotFoundException(event.applicationId()));

        // Compensation: khong reject, chuyen sang manual review
        app.addNote("SYSTEM",
            "CV auto-parse that bai: " + event.reason()
            + ". Can recruiter kiem tra thu cong.");

        applicationRepo.save(app);

        log.warn("Saga compensation: application={} needs manual CV review. Reason: {}",
                 event.applicationId(), event.reason());
    }
}
```

06

Xử lý failure và compensation

6.1. Bảng tổng hợp các kịch bản lỗi

KỊCH BẢN	BƯỚC LỖI	COMPENSATION	KẾT QUẢ CUỐI
Job đã expired	T1 (validate)	Không cần (chưa ghi gì vào DB)	Trả lỗi 400 cho client
Duplicate application	T1 (validate)	Không cần (chưa ghi gì vào DB)	Trả lỗi 409 cho client
Notification Service down	T2	Không cần compensation cho T1 (email là retriable, RabbitMQ retry/DLQ)	Email gửi trễ, application vẫn hợp lệ
CV file corrupted	T3 (parse)	C1: Application chuyển manual review. C2: Email thông báo recruiter	Application giữ stage NEW, recruiter xem xét thủ công
MinIO down (không download được CV)	T3	CV Parser retry 3 lần. Nếu vẫn fail: publish CVParseFailed	Như trên: manual review
Application Service crash sau T1, trước T4	T4	Khi restart, Kafka consumer re-consume CVParsed (offset chưa commit). Idempotent check	Saga tiếp tục từ T4

6.2. Idempotent consumer (nhắc lại)

Vì Saga dùng at-least-once delivery, mỗi consumer phải kiểm tra xem event đã được xử lý chưa trước khi thực hiện lại. Hai cách phổ biến:

- **Check business state:** Nếu `application.stage` đã là `CV_SCREENING`, bỏ qua `CVParsed` event.
- **Processed event table:** Lưu `eventId` vào một bảng `processed_events`. Trước khi xử lý, kiểm tra `eventId` đã tồn tại chưa.

```
@KafkaListener(topics = "cv.parsed", groupId = "application-saga-group")
@Transactional
public void onCvParsed(String message) {
    CVParsedEvent event = parseEvent(message, CVParsedEvent.class);

    Application app = applicationRepo.findById(event.applicationId())
        .orElseThrow();

    // Idempotent check: neu da o CV_SCREENING hoac cao hon, bo qua
    if (app.getCurrentStage() != PipelineStage.NEW) {
        log.info("Idempotent skip: application={} already at stage={}",
                 event.applicationId(), app.getCurrentStage());
        return;
    }

    app.advanceStage(PipelineStage.CV_SCREENING);
    applicationRepo.save(app);
}
```

07

Các lưu ý thiết kế Saga trong thực tế

7.1. Saga và Isolation

Saga không có property Isolation như ACID transaction. Điều này có nghĩa là các saga chạy đồng thời có thể thấy kết quả trung gian của nhau. Richardson đề xuất các countermeasure:

- **Semantic lock:** Đánh dấu trạng thái trung gian (ví dụ: `PENDING`) để ngăn saga khác tác động.
- **Commutative updates:** Thiết kế update sao cho thứ tự không ảnh hưởng kết quả.
- **Pessimistic view:** Sắp xếp các bước saga sao cho bước rủi ro nhất chạy sớm nhất (fail fast).
- **Re-read value:** Đọc lại giá trị trước khi quyết định, phát hiện thay đổi từ saga khác.

7.2. Observability cho Saga

Saga phân tán qua nhiều service, rất khó debug nếu không có observability tốt. Cần:

- **Correlation ID:** Đính kèm một ID duy nhất (ví dụ: `applicationId`) vào tất cả event và log. Khi debug, search theo ID này trên Grafana/Loki để thấy toàn bộ luồng.
- **Distributed tracing:** Zipkin/Jaeger trace request xuyên suốt các service. Mỗi bước saga là một span trong trace.
- **Saga state dashboard:** Dashboard hiển thị trạng thái saga (pending, completed, failed, compensating) để team vận hành theo dõi.

7.3. Khi nào dùng Saga, khi nào không cần

DÙNG SAGA	KHÔNG CẦN SAGA
Nghiệp vụ span nhiều service, mỗi service có database riêng	Tất cả logic nằm trong một service (dùng `@Transactional` thường)
Cần compensation khi bước trung gian thất bại	Bước sau chỉ là side effect không quan trọng (ví dụ: ghi log)
Yêu cầu eventual consistency chấp nhận được	Yêu cầu strong consistency tuyệt đối (ví dụ: chuyển tiền giữa hai tài khoản trong cùng ngân hàng)

08

Tóm tắt và câu hỏi ôn tập

Tóm tắt

KHÁI NIỆM	NỘI DUNG CHÍNH
Saga Pattern	Chuỗi local transaction, mỗi bước có compensating transaction tương ứng
Choreography	Không có coordinator; service communicate qua event
Orchestration	Có Saga Orchestrator điều phối; gửi command, đợi response
Compensating Transaction	Thao tác ngược để hoàn tác tác động của bước đã commit
Semantic Compensation	Không xóa dữ liệu mà chuyển sang trạng thái an toàn (manual review, pending)
Application Submission Saga	T1: tạo app + outbox, T2: email, T3: parse CV (pivot), T4: advance stage

Câu hỏi ôn tập

1. Giải thích tại sao 2PC không phù hợp với microservices. Saga giải quyết vấn đề này bằng cách nào?
2. So sánh Choreography và Orchestration Saga. Cho ví dụ cụ thể khi nào chọn choreography, khi nào chọn orchestration.
3. Vẽ compensation chain cho luồng Offer trong TalentHub: HM approve offer, gửi offer letter cho candidate, candidate chấp nhận. Nếu candidate từ chối, compensation chain là gì?
4. Giải thích khái niệm "pivot transaction". Trong Application Submission Saga, bước nào là pivot? Tại sao?
5. Tại sao Saga không có tính chất Isolation? Đề xuất ít nhất 2 countermeasure.
6. Một saga có 5 bước: T1, T2, T3 (pivot), T4, T5. T4 thất bại. Cần compensate những bước nào? Tại sao?

Bài tập nhóm (thảo luận tại lớp)

Mỗi nhóm chọn một luồng nghiệp vụ trong TalentHub và thiết kế Saga:

1. **Luồng Interview Scheduling:** Recruiter đặt lịch phỏng vấn. Application chuyển stage. Notification gửi email mời candidate. Calendar Service tạo event (giả lập). Nếu candidate từ chối lịch?
2. **Luồng Offer:** HM approve. Application tạo offer. Notification gửi offer letter. Candidate accept/decline. Compensation khi candidate decline?
3. **Luồng Bulk Rejection:** Recruiter reject nhiều application cùng lúc. Notification gửi email rejection hàng loạt. Nếu email service quá tải?

Yêu cầu: (1) vẽ sequence diagram happy path, (2) xác định pivot transaction, (3) vẽ compensation chain, (4) xác định bước nào cần idempotent.

Tài liệu tham khảo

1. Garcia-Molina, H., Salem, K. (1987). "Sagas". *Proceedings of the ACM SIGMOD International Conference on Management of Data.* cs.cornell.edu (PDF)
2. Richardson, C. (2018). *Microservices Patterns*, Ch. 4: Managing transactions with sagas. Manning. microservices.io/patterns/data/saga.html
3. Newman, S. (2021). *Building Microservices*, 2nd ed., Ch. 6: Workflow (Sagas). O'Reilly.
4. Larsson, M. (2023). *Microservices with Spring Boot 3 and Spring Cloud*, 3rd ed. Packt.
5. Kleppmann, M. (2017). *Designing Data-Intensive Applications*, Ch. 9: Consistency and Consensus. O'Reilly.
6. Baeldung: Saga Pattern in Microservices. baeldung.com/cs/saga-pattern-microservices